



IoT Village Hardware Hacking Exercises 2026

JTAG to Root

Summary:

The goal of this year's hands-on hardware hacking exercise will be to expose attendees to a new hardware attack vector and toolset, including OpenOCD, TIO, JLink, and live memory manipulation via JTAG. Attendees will use these tools and techniques to gain access to an OpenMesh A60 WiFi access point platform by temporarily modifying the Linux kernel boot arguments in RAM during the boot process, allowing the device to boot directly into single-user mode without making any permanent modifications to the flash memory. With single-user mode access, attendees will use a series of Linux commands to manually recreate the minimum operating environment required for the system to function, including mounting the required virtual file systems, creating essential device nodes, and restarting key system services. They will then manually load the Ethernet kernel driver, configure the network interface, and launch the Dropbear SSH server to restore remote administrative access to the device. By the end of the exercise, attendees will have transformed a minimal single-user shell into a functional embedded Linux environment while gaining a deeper understanding of the Linux boot process, runtime initialization, and the techniques used to recover and administer embedded systems without modifying persistent storage.

Within this exercise the attendee will be exposed to the following items of interest:

- Universal Asynchronous Receiver / Transmitter (UART)
- OpenOCD:
- Hexedit: view memory dumps
- Single-user mode and file system & network service recovery

Requirements	
Hardware	• Laptop • OpenMesh A60 WiFi AP • Tigard • Segger J-Link EDU Mini
Software	• Terminal • Hexedit • TIO Serial Terminal • OpenOCD

Documentation	• This exercise manual

I highly recommend reading ahead for each section before you start. The device has a watchdog and after a short period of time (5 Minutes) it will reboot, thinking the system applications have crashed. This watchdog timeout will impact each section 3, 4, and 5. Being ready for each section of the exercise will keep you from restarting over. Once the watchdog is started in section Step 5.3, you should be fine.

During this exercise there are many long commands and multiple commands that will need to be executed in quick succession. To make that operation easier and successful, we have created a text file available on your desktop called "Command Notes". You can use the commands listed in this file to cut and paste the longer commands needed during the exercise. Also note, commands listed one after another can all be cut and pasted all at the same time, just remember to hit enter after.

Also, during this exercise and after you have completed the exercise, **do not** disconnect any of the cables, wires or plugs.

Part 1 – System Boot and Evaluation

In this section of the exercise, you will first validate that the FTDI serial device (Tigard), Segger Jlink, Ethernet, and POE are all properly connected. Once that is done, we will power up the OpenMesh device and use UART serial connection to monitor the bootup process.



Figure 1: OpenMesh A60 WiFi Hardware

Note: The Tigard (Figure 2) is a hardware hacking tool designed by Joe FitzPatrick for interacting with and reverse-engineering embedded systems. It is based on the FTDI FT2232H chip, which provides various functionalities for communication and debugging and can be purchased online from various resources.

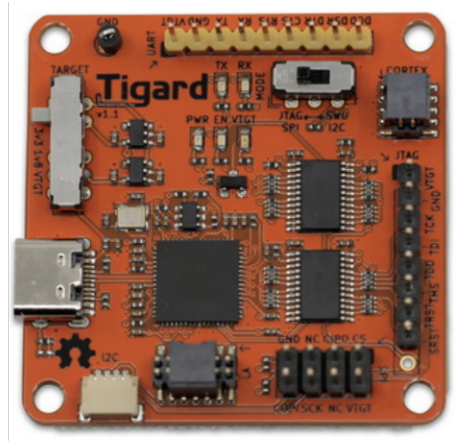


Figure 2: Tigard Debug Board

Step 1.1: First step is to make sure the following items are connected correctly as shown below in Figure 3:

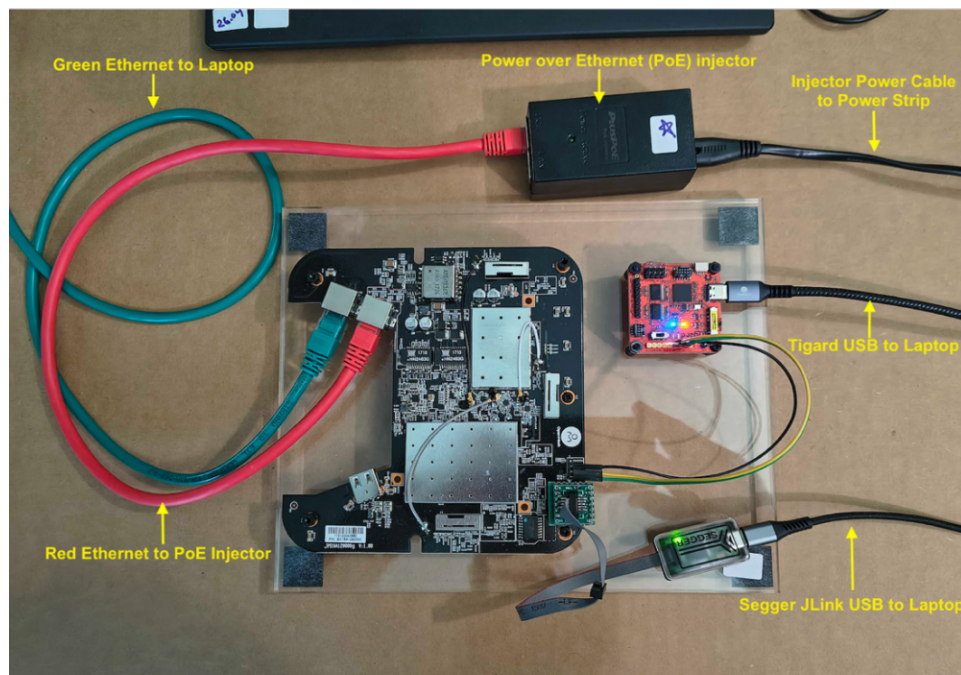


Figure 3: Initial Hardware Setup

- Tigard USB is connected to laptop
- The Segger Jlink is attached to the laptop via USB cable
- Red ethernet cable is attached to the PoE device and the inside ethernet socket
- PoE Injector is attached to power strip
- Green ethernet cable is attached to the outer ethernet socket and the laptop

Step 1.2: Next you need to start the serial terminal “tio” it should be preconfigured using a predefined config file. To do this, first open a Terminal by clicking on the terminal icon in the left column of the laptop desktop as shown in Figure 4:



Figure 4: Terminal Icon

Once terminal is open, you can launch the serial application “tio” by running the following command in the terminal to start the serial application using its default configuration Figure 5:

tio default

```
lab@lab-Rapid7-Lab-System:~/exercise/work$ tio default
[16:14:52.287] tio 3.9
[16:14:52.287] Press ctrl-t q to quit
[16:14:52.292] Connected to /dev/ttyUSB0
```

Figure 5: Launch tio

Note: tio was preconfigured using a configuration file located at ~/.config/tio/config. tio¹ can also be configured from the command line during its initial launch. We chose to use the config file method to simplify this exercise. Below is an example of the config file used on your laptop

```
[default]
device = /dev/ttyUSB0
baudrate = 115200
databits = 8
parity = none
stopbits = 1
flow = none
local-echo = false
timestamp = false
color = 177
log = false
```

Note: So, I expect you are also wondering how to control the tio console. Tio is easily controlled with some simple keyboard shortcuts. Here are several of the most common ones. The two I use all the time are in bold

¹ <https://github.com/tio/tio/blob/master/README.md>

Key commands:

<code>ctrl-t c</code>	Show configuration
<code>ctrl-t f</code>	Toggle log to file
<code>ctrl-t l</code>	Clear screen
<code>ctrl-t L</code>	Show line states
<code>ctrl-t q</code>	Quit
<code>ctrl-t s</code>	Show statistics
<code>ctrl-t t</code>	Toggle line timestamp mode
<code>ctrl-t v</code>	Show version
<code>ctrl-t ?</code>	List available key commands

Step 1.3: Once the hardware connections have been validated and tio is running, you can now power on the OpenMesh device by turning **ON** the power strip that the device is attached to.

When the OpenMesh device powers up, you should see activity in the tio terminal screen, similar to what is shown below in Figure 6:

```
U-Boot 1.1.4 (Mar 21 2017 - 13:15:34)

A60 (ar955x) U-boot
- Scorpion 1.0DRAM:
sri
Scorpion 1.0
ath_ddr_initial_config(177): (32bit) ddr2 init
tap = 0x00000003
Tap (low, high) = (0x2, 0x1f)
Tap values = (0x10, 0x10, 0x10, 0x10)
128 MB
Top of RAM usable for U-Boot at: 88000000
Reserving 237k for U-Boot at: 87fc4000
Reserving 192k for malloc() at: 87f94000
Reserving 44 Bytes for Board Info at: 87f93fd4
Reserving 36 Bytes for Global Data at: 87f93fb0
Reserving 128k for boot params() at: 87f73fb0
Stack Pointer at: 87f73f98
Now running in RAM - U-Boot at: 87fc4000
Flash Manuf Id 0xc2, DeviceId0 0x20, DeviceId1 0x18
flash size 16MB, sector count = 256
```

Figure 6: OpenMesh Booting

If nothing appears, something may be misconfigured. First, double-check all cable connections. If the issue persists, ask an exercise proctor for assistance.

Also, during the boot process, you will notice that the U-Boot console will display (**Hit any key to stop autoboot**) (Figure 7). This is where you see the device do a count down from 4. This is used to gain access to the U-Boot console for configuration of U-Boot settings. We have used this during past year's exercises to gain access to the device.

```

athr_gmac_sgmi_setup SGMI done
: cfg1 0x800c0000 cfg2 0x7214
eth1 up
eth0, eth1
Setting 0x18116290 to 0x4081a14f
Hit any key to stop autoboot: 3

```

Figure 7: BootDelay Counter

Step 1.4: Before this count down finishes, **hit the enter key** and watch what happens. If you missed it, go ahead and power off the device and start it over.

When you hit enter during the count down, it just reboots the system instead of starting a U-Boot console.

Do this several times and watch it restart!!! It just starts the boot over, as shown below in Figure 8:

```

athr_gmac_sgmi_setup SGMI done
: cfg1 0x800c0000 cfg2 0x7214
eth1 up
eth0, eth1
Setting 0x18116290 to 0x4081a14f
Hit any key to stop autoboot: 0
U-boot shell is disabled for this device

U-Boot 1.1.4 (Mar 21 2017 - 13:15:34)

A60 (ar955x) U-boot
- Scorpion 1.0DRAM:
sri
Scorpion 1.0
ath_ddr_initial_config(177): (32bit) ddr2 init

```

Figure 8: U-Boot Reboot

Step 1.5: After completing the above steps, just let the device count down and fully boot up. Once fully booted you should see it stabilize and slow to a halt, as shown below in Figure 9:

```

[ 20.164895] state 0x1
[ 20.177263] Reached count 1
[ 20.180100] eth1: link up (1000Mbps/Full duplex)
[ 20.214037] IPv6: ADDRCONF(NETDEV_CHANGE): eth1: link becomes ready
[ 20.220588] br-lan2: port 1(eth1) entered forwarding state
[ 20.226222] br-lan2: port 1(eth1) entered forwarding state
[ 31.929373] br-meship: port 1(bat0.3) entered forwarding state
[ 32.129374] br-ssid2: port 1(bat0.12) entered forwarding state
[ 32.169377] br-ssid1: port 1(bat0.11) entered forwarding state
[ 35.249381] br-lan2: port 1(eth1) entered forwarding state
[ 40.849411] random: nonblocking pool is initialized
[ 50.165737] batman_adv: bat0: Changing gw mode from: off to: client

```

Figure 9: Full System Boot

Once it's in this state, hit the enter key three times. It should show that the console access is disabled "shell disabled" Figure 10:

```
[ 32.129374] br-ssid2: port 1(bat0.12) entered forwarding state
[ 32.169377] br-ssid1: port 1(bat0.11) entered forwarding state
[ 35.249381] br-lan2: port 1(eth1) entered forwarding state
[ 40.849411] random: nonblocking pool is initialized
[ 50.165737] batman_adv: bat0: Changing gw mode from: off to: client

shell disabled..Please press Enter to activate this console.

shell disabled..Please press Enter to activate this console.

shell disabled..Please press Enter to activate this console.
```

Figure 10: Shell Disabled

Step 1.6: Next, turn the power off on the OpenMesh device by turning off the power strip that it is connected to. Once power is turned off, examine the output in the tio console. This can be done by using the mouse to scroll up and down to examine the boot process output. Here, we will point out a couple areas of interest which will come in handy later.

First, scroll to the beginning of the boot process, where it starts with **U-Boot 1.1.4**. If we scroll down from there, we come across a simple memory map being reported in the boot process. This data is invaluable when we start working on further attacks against the system. A sample of this is shown below in Figure 11:

```
U-Boot 1.1.4 (Mar 21 2017 - 13:15:34)

A60 (ar955x) U-boot
- Scorpion 1.0DRAM:
sri
Scorpion 1.0
ath_dds_initial_config(177): (32bit) ddr2 init
tap = 0x00000003
Tap (low, high) = (0x2, 0x1f)
Tap values = (0x10, 0x10, 0x10, 0x10)
128 MB
Top of RAM usable for U-Boot at: 88000000
Reserving 237k for U-Boot at: 87fc4000
Reserving 192k for malloc() at: 87f94000
Reserving 44 Bytes for Board Info at: 87f93fd4
Reserving 36 Bytes for Global Data at: 87f93fb0
Reserving 128k for boot params() at: 87f73fb0
Stack Pointer at: 87f73f98
Now running in RAM - U-Boot at: 87fc4000
Flash Manuf Id 0xc2, DeviceId0 0x20, DeviceId1 0x18
flash size 16MB, sector count = 256
Flash: 16 MB
*** Warning *** : PCIe WLAN Module not found !!!
In: serial
Out: serial
```

Figure 11: Boot Console Memory Map

Next, scroll down further and we find more valuable information. A sample of this is shown below in Figure 12. All this information will come in handy as we proceed through the exercise.

- 1) Certain file systems are signed with MD5 checksum. That means if you alter those, the hash won't match and it will not load.
- 2) The operating system used is MIPS OpenWRT Linux.
- 3) The system processor is a Qualcomm Atheros QCA9558 (Which is a MIPS processor).

```

Retry count exceeded; starting again
Retry
Validating MD5Sum of 'vmlinux'... Passed!
Validating MD5Sum of 'rootfs'... Passed!
## Booting image at 9f0b0000 ...
Image Name: MIPS OpenWrt Linux-4.4.184
Created: 1970-01-01 0:00:00 UTC
Image Type: MIPS Linux Kernel Image (lzma compressed)
Data Size: 1296372 Bytes = 1.2 MB
Load Address: 80060000
Entry Point: 80060000
Verifying Checksum at 0x9f0b0040 ...OK
Uncompressing Kernel Image ... OK
No initrd
## Transferring control to Linux (at address 80060000) ...
## Giving linux memsize in bytes, 134217728

Starting kernel ...

[ 0.000000] Linux version 4.4.184 (fw-builder@fw-builder) (gcc
951) ) #0 Tue Oct 15 18:00:28 UTC 2019
[ 0.000000] bootconsole [early0] enabled
[ 0.000000] CPU0 revision is: 00019750 (MIPS 74Kc)
[ 0.000000] SoC: Qualcomm Atheros QCA9558 ver 1 rev 0
[ 0.000000] Determined physical RAM map:

```

Figure 12: Usable Boot Information

Note: The MIPS (Microprocessor without Interlocked Pipeline Stages) processor is a Reduced Instruction Set Computing (RISC) architecture designed to provide high performance through a simplified instruction set. Although ARM processors have become the dominant architecture in most modern embedded systems, MIPS processors remain widely deployed in legacy and long-lived embedded devices such as WiFi routers, wireless access points, mesh networking hardware, cable and DSL modems, firewalls, IP cameras, industrial networking equipment, and many Internet of Things (IoT) devices. As a result, security researchers and hardware hackers continue to encounter MIPS-based systems during firmware analysis, vulnerability research, and hardware reverse engineering. The WiFi mesh hardware used in this exercise is powered by a Qualcomm Atheros QCA9558 MIPS processor, providing attendees with hands-on experience analyzing and interacting with a MIPS-based embedded Linux system using JTAG and OpenOCD.

Part 2 – JTAG & OpenOCD Setup

In the previous sections we discovered that U-Boot console access is blocked, the system just reboots when we hit return to stop the boot process, and the shell console is disabled. Since we are currently blocked from either modifying the U-Boot environment or accessing the UART console, we will need to find another way to gain access to the system and control it. To address this, we will utilize JTAG to establish a connection to the system microprocessor (Qualcomm Atheros QCA9558 MIPS processor), along with data discovered in Part 1, to target the system to gain access.

Step 2.1: The first thing we want to do is start several terminals to handle various parts of the JTAG/OpenOCD parts of the exercise. To do this, first open a Terminal by right clicking on terminal and select **New Window**, as shown in Figure 13:

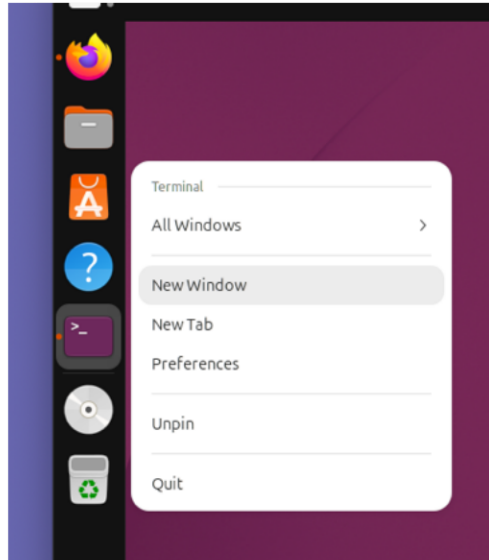


Figure 13: New Terminal Window

Step 2.2: Once that terminal is open, click on it and use the **CTRL-SHIFT-T** keys to open two more terminals for a total of three, as shown below Figure 14:

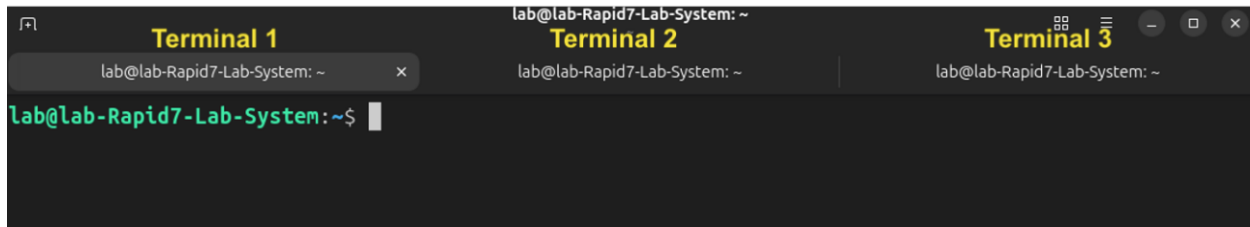


Figure 14: Terminal Layout Order

In the following steps, we will start OpenOCD, which we will use along with the Segger J-Link EDU Mini hardware debugger to establish a JTAG connection to the OpenMesh QCA9558 MIPS processor. But before we start, let's first define some basic understanding of the application and hardware being used to do those steps:

Note: JTAG (Joint Test Action Group) is a standardized hardware interface originally developed for manufacturing and board-level testing, but it has become one of the most valuable interfaces for embedded system debugging and hardware security research. JTAG provides direct access to a processor's debug functions, allowing developers and researchers to halt the processor, inspect and modify registers, read and write system memory, set breakpoints, and control program execution independently of the operating system. Because JTAG operates below the software layer, it can often be used to analyze and recover systems even when the operating system is inaccessible or security controls have been enabled. In this exercise, attendees will use the JTAG

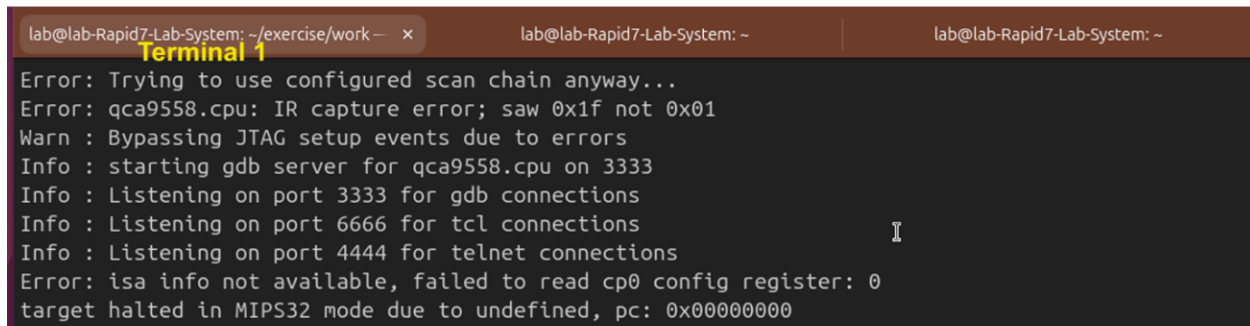
interface to establish a direct hardware connection to the OpenMesh's MIPS processor, enabling OpenOCD to temporarily modify the Linux kernel boot arguments in RAM and obtain single-user access without making any permanent changes to the device's flash memory.

Note: OpenOCD (Open On-Chip Debugger) is an open-source debugging tool that communicates with embedded processors through hardware debug interfaces such as JTAG and Serial Wire Debug (SWD). It provides direct access to a target system's processor and memory, allowing researchers to halt execution, inspect and modify registers, read and write memory, set breakpoints, and resume program execution.

Note: The SEGGER J-Link EDU Mini is a compact USB JTAG/SWD debug probe designed for educational and non-commercial use. It provides a reliable hardware interface between a host computer and the target embedded processor, allowing debugging tools such as OpenOCD to communicate with the device over its JTAG interface. In this exercise, the J-Link EDU Mini is used to establish a hardware debugging connection to the target's MIPS processor, enabling attendees to halt the processor, inspect and modify system memory, and resume execution.

Step 2.3: Next, we will launch OpenOCD. First, select Terminal 1 of the three you started. In Terminal 1, enter the following command shown below. If prompted for a password use **toor**. You may receive some error messages. This is common, since the OpenMesh device is currently powered down. Once OpenOCD is started it should look similar to Figure 15:

sudo openocd -f ~/exercise/ocd_config/openocd.cfg

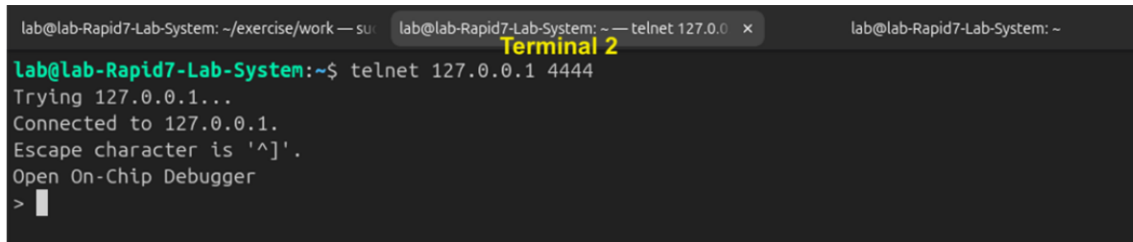
A screenshot of a terminal window titled 'Terminal 1' with a dark background. The terminal shows the output of the 'sudo openocd' command. The output includes several error and warning messages, followed by information about starting the GDB server and listening on ports 3333, 6666, and 4444. The terminal ends with a message indicating the target is halted in MIPS32 mode due to an undefined PC value.

```
lab@lab-Rapid7-Lab-System: ~/exercise/work — x lab@lab-Rapid7-Lab-System: ~ lab@lab-Rapid7-Lab-System: ~
Terminal 1
Error: Trying to use configured scan chain anyway...
Error: qca9558.cpu: IR capture error; saw 0x1f not 0x01
Warn : Bypassing JTAG setup events due to errors
Info : starting gdb server for qca9558.cpu on 3333
Info : Listening on port 3333 for gdb connections
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
Error: isa info not available, failed to read cp0 config register: 0
target halted in MIPS32 mode due to undefined, pc: 0x00000000
```

Figure 15: OpenOCD Launch

Step 2.4: Next, to access the OpenOCD application select Terminal 2 and run the following telnet command to connect to OpenOCD (Figure 16).

telnet 127.0.0.1 4444



```
lab@lab-Rapid7-Lab-System: ~/exercise/work — su: lab@lab-Rapid7-Lab-System: ~ — telnet 127.0.0.1 x lab@lab-Rapid7-Lab-System: ~
Terminal 2
lab@lab-Rapid7-Lab-System:~$ telnet 127.0.0.1 4444
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
Open On-Chip Debugger
>
```

Figure 16: Telnet to OpenOCD Console

Note: When we launched the OpenOCD we also pointed it to “~/exercise/ocd_config/openocd.cfg” using the -f switch , let's take a brief look at the config files contents and understand the role it plays in establishing a debugging session. The OpenOCD configuration file defines how OpenOCD communicates with the target hardware by specifying the debug adapter (jlink), selecting the transport (jtag), configuring the communication speed (1000) and reset behavior, and identifying the target as a Qualcomm Atheros QCA9558 MIPS processor (qca9558). It also creates the processor debug target used by OpenOCD and registers the onboard SPI flash device so it can be accessed during the debugging session if needed. Finally, the (init) command establishes the JTAG connection and prepares the target for debugging and memory access. The complete configuration file used throughout this exercise is shown below.

```
adapter driver jlink
transport select jtag
adapter speed 1000
reset_config trst_and_srst
adapter srst delay 200
jtag_ntrst_delay 200
set _CHIPNAME qca9558
jtag newtap $_CHIPNAME cpu -irlen 5 -ignore-version
set _TARGETNAME $_CHIPNAME.cpu
target create $_TARGETNAME mips_m4k -endian big -chain-position
$_TARGETNAME
# Declare the SPI flash via ath79 BEFORE init
flash bank sysflash ath79 0 0 0 0 $_TARGETNAME
init
```

Part 3 – JTAG Attachment and Memory Dump.

Now that OpenOCD is running, we will use information we gathered during the boot process to target memory and extract and locate data that can be altered during the boot process to force the device into a single user mode.

The first thing we need to think about, is where do we halt the system so the memory state will always be in the same static state allowing us to extract and alter memory and make that repeatable. Well, we are going to cheat. Remember when we watched the boot process, and I pointed out the counting down. Well, at that point U-Boot is in memory and it's not being changed, and the kernel has not been loaded yet, the system is just cycling a counter. So, what we are after should be in a fixed location in memory, even if we restart the system over and over. This makes for a perfect repeatable process. Second, the memory map we examined during the boot told us where U-Boot was loaded in memory and where the memory allocation

area was. Both of these (memory map & count down) are shown below (Figure 17) as a reminder, and we are going to use them.

```
Top of RAM usable for U-Boot at: 88000000
Reserving 237k for U-Boot at: 87fc4000
Reserving 192k for malloc() at: 87f94000
Reserving 44 Bytes for Board Info at: 87f93fd4

athr_gmac_sgmi_setup SGMII done
: cfg1 0x800c0000 cfg2 0x7214
eth1 up
eth0, eth1
Setting 0x18116290 to 0x4081a14f
Hit any key to stop autoboot: 3
```

Figure 17: Memory Map and U-Boot Delay Count Down

Step 3.1: So, what we are going to do is power up the OpenMesh device, run **reset** in OpenOCD so it reconnects to the MIPS processor, and then we will run **halt** during the boot countdown. From there we are going to make a dump of a section of allocated memory and search for bootargs data strings, examine them, and determine the offset address so we can target that area of memory for alteration. First, let's orient our desktop so we can control OpenOCD and at the same time see the device booting. To do this, arrange your desktop to look something like Figure 18, with tio in lower right corner and 3 terminals in upper left corner.

- If tio is showing data, you can clear it with **CTRL-T L**
- After clearing it, make sure you click into the Terminal 2 console which is telnet'd into OpenOCD

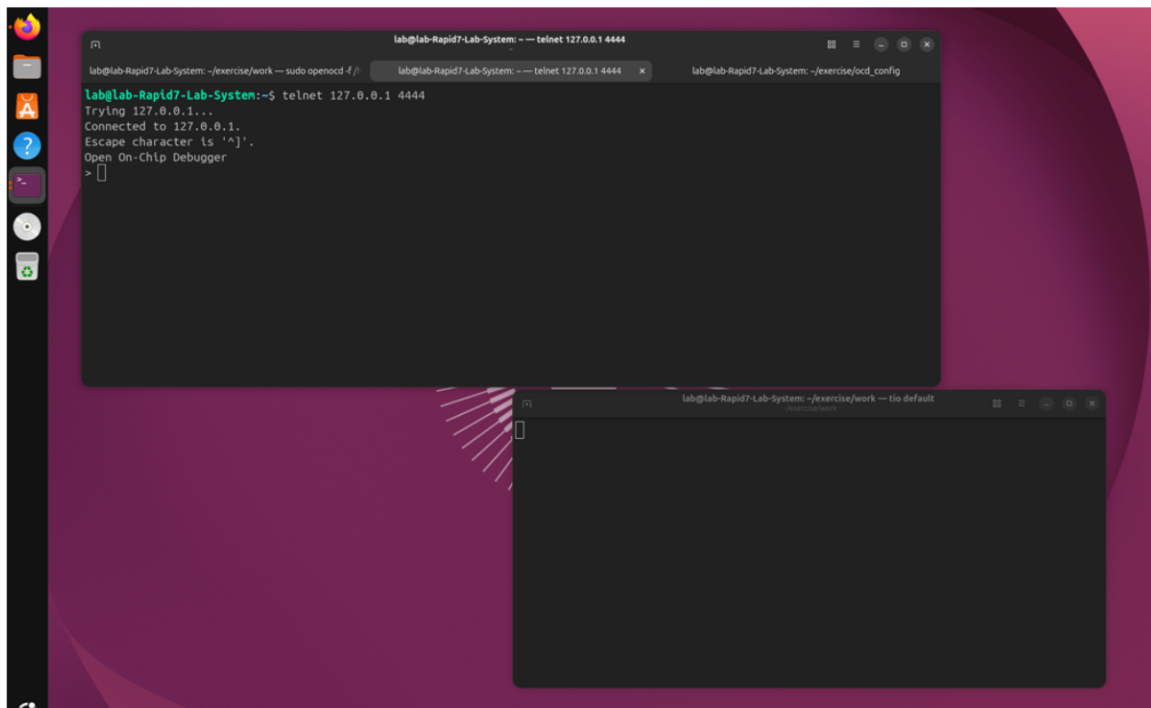


Figure 18: Desktop Orientation

Step 3.2: To establish a JTAG connection and halt the processor, you will need to do the following in order:

- 1) Enter **reset** in the telnet Terminal 2 – **Do Not Hit Enter**

- 2) Power on the device using the power strip
- 3) When you see activity in the tio console – hit return key – this will execute **reset**
- 4) Quickly enter **halt** in the telnet Terminal 2 – Hit Enter when you see the countdown start
- 5) If you missed the countdown, you will need to restart Step 3.2 after powering off your device

If you were successful, you should see the following state of the system, showing that the count down has halted (Figure 19).

The figure consists of two terminal windows. The top window is a telnet session to 127.0.0.1 4444. It shows the following commands and output:

```
lab@lab-Rapid7-Lab-System:~$ telnet 127.0.0.1 4444
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
Open On-Chip Debugger
> reset
JTAG tap: qca9558.cpu tap/device found: 0x00000001 (mfg: 0x000 (<invalid>), part: 0x0000, ver: 0x0)
> halt
MIPS32 with MIPS16 support implemented
target halted in MIPS32 mode due to debug-request, pc: 0x87fda41c
>
```

Annotations on the top window: A yellow arrow points to the 'reset' command with the text 'reset established connection'. A red arrow points to the 'halt' command with the text 'halt, halted the CPU'.

The bottom window is a 'tio default' terminal showing the system boot logs:

```
lab@lab-Rapid7-Lab-System:~/exercise/work -- tio default
Fetching MAC Address from 8033 0x87fed0cc
ath_gmac_enet_initialize: reset mask:c02200
Scorpion ----> S17 PHY *
athrs17_reg_init: complete
: cfg1 0x80000000 cfg2 0x7135
eth0 up
Scorpion ----> 8033 PHY*
AR8033 PHY init
athrs_ar8033_reg_init: Done ffff
Max resets limit reached exiting...
athr_gmac_sgmi_setup SGMII done
: cfg1 0x800c0000 cfg2 0x7214
eth1 up
eth0, eth1
Setting 0x18116290 to 0x4081a14f
Hit any key to stop autoboot: 3
```

Annotation on the bottom window: A green arrow points to the 'Hit any key to stop autoboot: 3' line with the text 'CPU halted during the countdown'.

Figure 19: OpenMesh MIPS Processor Halted

Step 3.3: Once halted, we can use the following OpenOCD command to dump a section of memory area. To save time, we have reduced this down to only look at a small portion of MALLOC() area.

dump_image uboot_malloc.bin 0x87f94000 0x00001000

When this command is executed in OpenOCD, you should see the following (Figure 20) in Terminal 2.

```

lab@lab-Rapid7-Lab-System:~$ telnet 127.0.0.1 4444
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
Open On-Chip Debugger
target halted in MIPS32 mode due to debug-request, pc: 0x87fda41c
> reset
JTAG tap: qca9558.cpu tap/device found: 0x00000001 (mfg: 0x000 (<invalid>), part: 0x0000, ver: 0x0)
> halt
target halted in MIPS32 mode due to debug-request, pc: 0x87fda41c
> dump_image uboot_malloc.bin 0x87f94000 0x00001000
dumped 4096 bytes in 2.791945s (1.433 KiB/s)
>

```

Figure 20: OpenOCD memory dump

Also, if you click on the Terminal 3 and enter the following command, you should see the file containing the memory dump (Figure 21).

ls -al

```

lab@lab-Rapid7-Lab-System:~/exercise/work$ ls -al
total 12
drwxrwxr-x 2 lab lab 4096 Jun 30 16:32 .
drwxrwxr-x 5 lab lab 4096 Jun 29 13:00 ..
-rw-r--r-- 1 root root 4096 Jun 30 16:32 uboot_malloc.bin
lab@lab-Rapid7-Lab-System:~/exercise/work$

```

Figure 21: File & Directory List command

At this point, you can turn the power off on the OpenMesh device by turning off the power strip, and we will address in more detail what we just did. We will examine the memory dump to identify our target and offset for changing memory.

Note: The reason we searched near 0x87f94000 is that U-Boot boot logs showed us in the memory map it had reserved memory near the top of RAM for its relocated runtime, malloc area, and boot parameters. Since U-Boot constructs the kernel command line before transferring execution to Linux, the boot arguments are expected to exist as ASCII strings in one of those RAM areas. Instead of scanning the entire memory address space, we used U-Boot's memory reservation output to focus on the 128 MB high-RAM region around 0x87f00000-0x88000000, then searched for known boot argument strings we could target. For this part of the exercise, I narrowed down the memory dump to a very small area, where I knew our target existed, to speed up the exercise.

Step 3.4: Next, let's open the memory dump file we created with the tool hexedit for exploring it and identify the offset of our target data. To open **uboot_malloc.bin**, select Terminal 3 and run the following command within the terminal. Once hexedit is started it should look similar to Figure 22:

hexedit uboot_malloc.bin

```
lab@lab-Rapid7-Lab-System: ~/exercise/work — sudo open: lab@lab-Rapid7-Lab-System: ~ — telnet 127.0.0.1 4444 lab@lab-Rapid7-Lab-System: ~/exercise/work — hexedit x
00000000 00 00 00 00 00 01 00 09 CB 82 46 13 62 6F 6F 74 61 72 67 73 .....F.bootargs
00000014 3D 63 6F 6E 73 6F 6C 65 3D 74 74 79 53 30 2C 31 31 35 32 30 =console=ttyS0,11520
00000028 30 20 72 6F 6F 74 66 73 74 79 70 65 3D 73 71 75 61 73 68 66 0 rootfstype=squashf
0000003C 73 2C 6A 66 66 73 32 20 69 6E 69 74 3D 2F 65 74 63 2F 70 72 s,jffs2 init=/etc/pr
00000050 65 69 6E 69 74 20 62 6F 61 72 64 3D 41 36 30 00 62 6F 6F 74 einit board=A60.boot
00000064 63 6D 64 3D 74 65 73 74 20 2D 6E 20 22 24 7B 70 72 65 62 6F cmd=test -n "${prebo
00000078 6F 74 7D 22 20 26 26 20 72 75 6E 20 70 72 65 62 6F 6F 74 3B ot}" && run preboot;
0000008C 20 74 65 73 74 20 2D 6E 20 22 24 7B 62 6F 6F 74 73 65 71 7D test -n "${bootseq}
000000A0 22 20 7C 7C 20 62 6F 6F 74 73 65 71 3D 31 2C 32 3B 20 72 75 " || bootseq=1,2; ru
000000B4 6E 20 73 65 74 5F 62 6F 6F 74 61 72 67 73 5F 31 3B 20 72 75 n set_bootargs_1; ru
000000C8 6E 20 73 65 74 5F 62 6F 6F 74 61 72 67 73 5F 32 3B 20 62 6F n set_bootargs_2; bo
000000DC 6F 74 20 22 24 7B 62 6F 6F 74 73 65 71 7D 22 00 62 6F 6F 74 ot "${bootseq}".boot
000000F0 64 65 6C 61 79 3D 34 00 62 61 75 64 72 61 74 65 3D 31 31 35 delay=4.baudrate=115
00000104 32 30 30 00 69 70 61 64 64 72 3D 31 39 32 2E 31 36 38 2E 31 200.ipaddr=192.168.1
00000118 30 30 2E 39 00 73 65 72 76 65 72 69 70 3D 31 39 32 2E 31 36 00.9.serverip=192.16
0000012C 38 2E 31 30 30 2E 38 00 70 72 65 62 6F 6F 74 3D 66 6C 61 73 8.100.8.preboot=flas
00000140 68 69 74 20 30 78 38 30 31 30 30 30 30 30 20 66 77 75 70 67 hit 0x80100000 fwupg
00000154 72 61 64 65 2E 63 66 67 00 69 6D 61 67 65 63 68 6B 3D 74 65 rade.cfg.imagechk=te
--% uboot_malloc.bin --0x0/0x1000--0%-----
```

Figure 22: uboot_malloc.bin Opened in hexedit

Before going any further we need to discuss a few basic functions about moving around in hexedit.

Using the **arrow keys**, you should be able to move the cursor.



Also, you can switch between hex bytes within the center columns, and Ascii on the right-side column and back and forth using the **Tab key**. These sections are annotated below in Figure 23:

```
lab@lab-Rapid7-Lab-System: ~/exercise/work — sudo open: lab@lab-Rapid7-Lab-System: ~ — telnet 127.0.0.1 4444 lab@lab-Rapid7-Lab-System: ~/exercise/work — hexedit x
00000000 00 00 00 00 00 01 00 09 CB 82 46 13 62 6F 6F 74 61 72 67 73 .....F.bootargs
00000014 3D 63 6F 6E 73 6F 6C 65 3D 74 74 79 53 30 2C 31 31 35 32 30 =console=ttyS0,11520
00000028 30 20 72 6F 6F 74 66 73 74 79 70 65 3D 73 71 75 61 73 68 66 0 rootfstype=squashf
0000003C 73 2C 6A 66 66 73 32 20 69 6E 69 74 3D 2F 65 74 63 2F 70 72 s,jffs2 init=/etc/pr
00000050 65 69 6E 69 74 20 62 6F 61 72 64 3D 41 36 30 00 62 6F 6F 74 einit board=A60.boot
00000064 63 6D 64 3D 74 65 73 74 20 2D 6E 20 22 24 7B 70 72 65 62 6F cmd=test -n "${prebo
00000078 6F 74 7D 22 20 26 26 20 72 75 6E 20 70 72 65 62 6F 6F 74 3B ot}" && run preboot;
0000008C 20 74 65 73 74 20 2D 6E 20 22 24 7B 62 6F 6F 74 73 65 71 7D test -n "${bootseq}
000000A0 22 20 7C 7C 20 62 6F 6F 74 73 65 71 3D 31 2C 32 3B 20 72 75 " || bootseq=1,2; ru
000000B4 6E 20 73 65 74 5F 62 6F 6F 74 61 72 67 73 5F 31 3B 20 72 75 n set_bootargs_1; ru
000000C8 6E 20 73 65 74 5F 62 6F 6F 74 61 72 67 73 5F 32 3B 20 62 6F n set_bootargs_2; bo
000000DC 6F 74 20 22 24 7B 62 6F 6F 74 73 65 71 7D 22 00 62 6F 6F 74 ot "${bootseq}".boot
000000F0 64 65 6C 61 79 3D 34 00 62 61 75 64 72 61 74 65 3D 31 31 35 delay=4.baudrate=115
00000104 32 30 30 00 69 70 61 64 64 72 3D 31 39 32 2E 31 36 38 2E 31 200.ipaddr=192.168.1
00000118 30 30 2E 39 00 73 65 72 76 65 72 69 70 3D 31 39 32 2E 31 36 00.9.serverip=192.16
0000012C 38 2E 31 30 30 2E 38 00 70 72 65 62 6F 6F 74 3D 66 6C 61 73 8.100.8.preboot=flas
00000140 68 69 74 20 30 78 38 30 31 30 30 30 30 30 20 66 77 75 70 67 hit 0x80100000 fwupg
00000154 72 61 64 65 2E 63 66 67 00 69 6D 61 67 65 63 68 6B 3D 74 65 rade.cfg.imagechk=te
--% uboot_malloc.bin --0x0/0x1000--0%-----
```

Figure 23: Hexedit Console

Step 3.5: So, for our next step, we need to locate the target in the memory dump. The first question here would be, what is the target? The answer is, we are going to target `init=` which currently points to `/etc/preinit` within the bootargs. We will be changing this to `init=/bin/sh`. A deeper explanation is available in the following note:

NOTE: The Linux kernel uses the `init=` boot parameter to determine which program is executed as the first user-space process after the kernel has initialized the hardware. Under normal operation, the OpenMesh device starts `/etc/preinit`, which continues the standard boot sequence by initializing the operating environment, mounting the writable overlay file system, starting system services, and eventually bringing the device into multi-user mode. For this exercise, we will temporarily be changing this boot parameter from `init=/etc/preinit` to `init=/bin/sh`, causing the kernel to launch a root shell immediately after boot. This will give us control over a single-user environment for analysis and recovery while avoiding the normal boot process. Because this modification is made only in RAM, it is temporary and does not permanently alter the device or its firmware.

To locate the offset address, you will need to first switch your cursor so it is on the Ascii side of the screen on the right (**Tab key**). Once you have done that, hold down the **Control key** and hit the **s key** (CTRL+s) at the same time. This should put you in the Ascii search mode, as shown below in Figure 24:

```
00000050  65 69 6E 69 74 20 62 6F 61 72 64 3D 41 36 30 00 62 6F 6F 74  einit board=A60.boot
00000064  63 6D 64 3D 74 65 73 74 20 2D 6E 20 22 24 7B 70 72 65 62 6F  cmd=test -n "${prebo
00000078  6F 74 7D 22 20 26 26 20 72 75 6E 20 70 72 65 62 6F 6F 74 3B  ot}" && run preboot;
0000008C  20 74 65 73 74 20 2D 6E 20 22 24 7B 62 6F 6F 74 73 65 71 7D  test -n "${bootseq}

                                Ascii string to search:

000000DC  6F 74 20 22 24 7B 62 6F 6F 74 73 65 71 7D 22 00 62 6F 6F 74  ot "${bootseq}".boot
000000F0  64 65 6C 61 79 3D 34 00 62 61 75 64 72 61 74 65 3D 31 31 35  delay=4.baudrate=115
00000104  32 30 30 00 69 70 61 64 64 72 3D 31 39 32 2E 31 36 38 2E 31  200.ipaddr=192.168.1
```

Figure 24: hexedit Ascii Search Mode

Step 3.6: Once the Ascii search prompt comes up, enter `preinit` and hit enter as shown below in Figure 25. This will take you to the beginning of the word `preinit`. In this case it is near the top of the file.

```
00000014  3D 63 6F 6E 73 6F 6C 65 3D 74 74 79 53 30 2C 31 31 35 32 30  =console=ttyS0,11520
00000028  30 20 72 6F 6F 74 66 73 74 79 70 65 3D 73 71 75 61 73 68 66  0 rootfstype=squashf
0000003C  73 2C 6A 66 66 73 32 20 69 6E 69 74 3D 2F 65 74 63 2F 70 72  s,jffs2 init=/etc/pr
00000050  65 69 6E 69 74 20 62 6F 61 72 64 3D 41 36 30 00 62 6F 6F 74  einit board=A60.boot
00000064  63 6D 64 3D 74 65 73 74 20 2D 6E 20 22 24 7B 70 72 65 62 6F  cmd=test -n "${prebo
00000078  6F 74 7D 22 20 26 26 20 72 75 6E 20 70 72 65 62 6F 6F 74 3B  ot}" && run preboot;
0000008C  20 74 65 73 74 20 2D 6E 20 22 24 7B 62 6F 6F 74 73 65 71 7D  test -n "${bootseq}
```

Figure 25: Searching For preinit String

Step 3.7: Next, we need to find the offset address of the starting point of string data we plan to change in running memory. In this case we can see `init=/etc/preinit`. So, what we need to locate is the first character of the string to be changed, `etc/preini`, in this case that would be `e`. So, using the arrow keys on the keyboard move the cursor to the beginning of `etc`. and note the offset address, an example of this is shown below in Figure 26:

```

00000000  00 00 00 00 00 01 00 09 CB 82 46 13 62 6F 6F 74 61 72 67 73 .....F.bootargs
00000014  3D 63 6F 6E 73 6F 6C 65 3D 74 74 79 53 30 2C 31 31 35 32 30 =console=ttyS0,11520
00000028  30 20 72 6F 6F 74 66 73 74 79 70 65 3D 73 71 75 61 73 68 66 0 rootfstype=squashf
0000003C  73 2C 6A 66 66 73 32 20 69 6E 69 74 3D 2F 65 74 63 2F 70 72 s,jffs2 init=/etc/pr
00000050  65 69 6E 69 74 20 62 6F 61 72 64 3D 41 36 30 00 62 6F 6F 74 einit board=A60.
00000064  63 6D 64 3D 74 65 73 74 20 2D 6E 20 22 24 7B 70 72 65 62 6F cmd=test -n "${prebo
00000078  6F 74 7D 22 20 26 26 20 72 75 6E 20 70 72 65 62 6F 6F 74 3B ot}" && run preboot;
0000008C  20 74 65 73 74 20 2D 6E 20 22 24 7B 62 6F 6F 74 73 65 71 7D test -n "${bootseq}
000000A0  22 20 7C 7C 20 62 6F 6F 74 73 65 71 3D 31 2C 32 3B 20 72 75 " || bootseq=1,2; ru
000000B4  6E 20 73 65 74 5F 62 6F 6F 74 61 72 67 73 5F 31 3B 20 72 75 n set_bootargs_1; ru
000000C8  6E 20 73 65 74 5F 62 6F 6F 74 61 72 67 73 5F 32 3B 20 62 6F n set_bootargs_2; bo
000000DC  6F 74 20 22 24 7B 62 6F 6F 74 73 65 71 7D 22 00 62 6F 6F 74 ot "${bootseq}".boot
000000F0  64 65 6C 61 79 3D 34 00 62 61 75 64 72 61 74 65 3D 31 31 35 delay=4.baudrate=115
00000104  32 30 30 00 69 70 61 64 64 72 3D 31 39 32 2E 31 36 38 2E 31 200.ipaddr=192.168.1
00000118  30 30 2E 39 00 73 65 72 76 65 72 69 70 3D 31 39 32 2E 31 36 00.9.serverip=192.16
0000012C  38 2E 31 30 30 2E 38 00 70 72 65 62 6F 6F 74 3D 66 6C 61 73 8.100.8.preboot=flas
00000140  68 69 74 20 30 78 38 30 31 30 30 30 30 30 20 66 77 75 70 67 hit 0x80100000 fwupg
00000154  72 61 64 65 2E 63 66 67 00 69 6D 61 67 65 63 68 6B 3D 74 65 rade.cfg.imagechk=te
-%% uboot_malloc.bin --0x4A/0x1000--2%-----

```

Figure 26: Locating Offset Address

Once you complete finding the memory offset of our target “etc/preinit” we can exit hexedit by holding down the **Control key** and hitting the **x key** (CTRL+x) at the same time.

Next we can do some basic math by adding that offset number to the beginning address of the U-boot Memory Allocation area malloc() which we identified by monitoring the system boot (Figure 27) and is also the beginning address of the memory dump we performed.

Reserving 192k for malloc() at: 87f94000

Figure 27: malloc() Memory Address

$$\begin{array}{r}
 0x87f94000 \\
 + \quad 0x4a \\
 \hline
 = 0x87f9404a
 \end{array}$$

Now, we can use this address 0x87f9404a within OpenOCD to overwrite the RAM memory on the OpenMesh device. This will cause the device to execute /bin/sh after boot, giving us a root shell

Part 4 – Time For a Root Shell

In this section of the exercise, we will take the information we gathered in previous sections and leverage OpenOCD to establish a JTAG connection and modify the entry **init=/etc/preinit** within the bootargs at the memory allocation area of RAM 0x87f9404a.

Step 4.1: First thing, we are going to repeat the steps from section 3.1-3.2 to gain JTAG access and halt the CPU. So first, let’s orient our desktop again so we can control OpenOCD and at same time see the device booting. To do this, re-arrange your desktop, if needed, to look something like this (Figure 28), with tio in the lower right corner and telnet in the upper left corner.

- If tio is showing data, you can clear it with **CTRL-T L**

- After clearing it, make sure you click into the Terminal 2 which is telnet'd into OpenOCD.

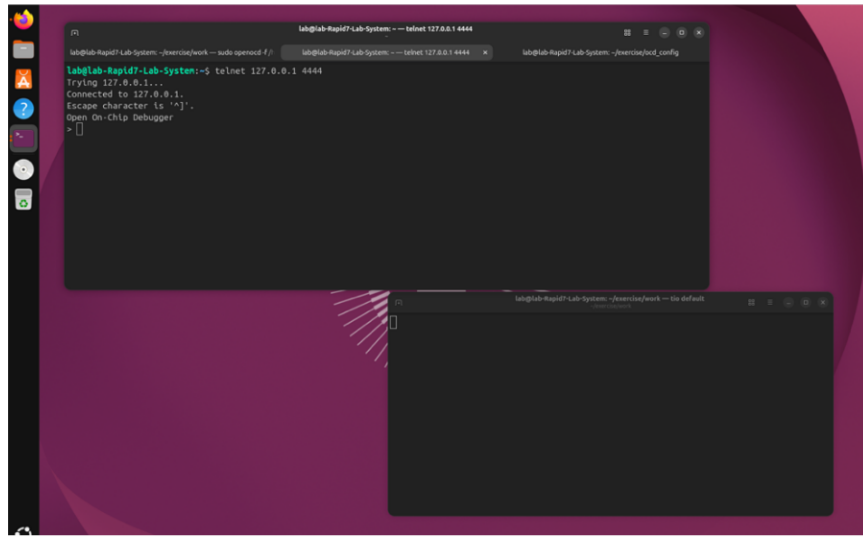


Figure 28: Re-arrange Desktop

Step 4.2: To establish a JTAG connection and halt the processor you will need to do the following in order:

- 1) Enter **reset** in the OpenOCD Telnet (Terminal 2) – **Do Not Hit Enter**
- 2) Power on the device using the power strip
- 3) When you see activity in the tio console – hit return key – this will execute **reset**
- 4) Quickly enter **halt** in the OpenOCD Telnet (Terminal 2) – Hit Enter when you see the countdown start
- 5) If you missed the countdown, you will need to restart this Step 4.2 after powering off your device

If you were successful, you should see the following state of the system Figure 29.

```

lab@lab-Rapid7-Lab-System: ~ -- telnet 127.0.0.1 4444
lab@lab-Rapid7-Lab-System: ~/exercise/work -- sudo openocd -f /
lab@lab-Rapid7-Lab-System: ~ -- telnet 127.0.0.1 4444
lab@lab-Rapid7-Lab-System: ~/exercise/ocd_config

lab@lab-Rapid7-Lab-System:~$ telnet 127.0.0.1 4444
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
Open On-Chip Debugger
> reset
JTAG tap: qca9558.cpu tap/device found: 0x00000001 (nfg: 0x000 (<invalid>), part: 0x0000, ver: 0x0)
> halt
MIPS32 with MIPS16 support implemented
target halted in MIPS32 mode due to debug-request, pc: 0x87fda41c
>

lab@lab-Rapid7-Lab-System: ~/exercise/work -- tio default
Fetching MAC Address from 8033 0x87fed0cc
ath_gmac_enet_initialize: reset mask:c02200
Scorpion ----> S17 PHY *
athrs17_reg_init: complete
: cfg1 0x80000000 cfg2 0x7135
eth0 up
Scorpion ---->8033 PHY*
AR8033 PHY init
athrs_ar8033_reg_init: Done ffff
Max resets limit reached exiting...
athr_gmac_sgmit_setup SGMII done
: cfg1 0x800c0000 cfg2 0x7214
eth1 up
eth0, eth1
Setting 0x18116290 to 0x4001a14f
Hit any key to stop autoboot: 3

```

Figure 29: OpenMesh MIPS Processor Halted

Remember, you have less than 5 minutes to finish the following section to gain root access in single user mode. If you don't succeed, the hardware watchdog will trigger the system to reboot and you will have to start over at step 4.2 over – Good Luck ;)

Remember to use the “Command Notes” file containing commands, to cut and paste the needed commands.

Step 4.3: Once the CPU has been properly halted, I want you to run the following command in Terminal 3 . This is a Linux command that will connect to OpenOCD and read the memory area at 0x87f94044, just short of our targeted address, and convert it from Hex to ascii and display it (Figure 30). We are doing these so you can see the results in standard ascii, versus trying to read hex.

printf "mdb 0x87f94044 18\nexit\n" | nc localhost 4444 | awk -F: '/0x/ {print \$2}' | xxd -r -p

```

lab@lab-Rapid7-Lab-System: ~/exercise/work -- sudo ope
lab@lab-Rapid7-Lab-System: ~ -- telnet 127.0.0.1 4444
lab@lab-Rapid7-Lab-System: ~/exercise/work

lab@lab-Rapid7-Lab-System:~/exercise/work$ printf "mdb 0x87f94044 18\nexit\n" | nc localhost 4444 | awk -F:
'/0x/ {print $2}' | xxd -r -p
init=/etc/preinit lab@lab-Rapid7-Lab-System:~/exercise/work$

```

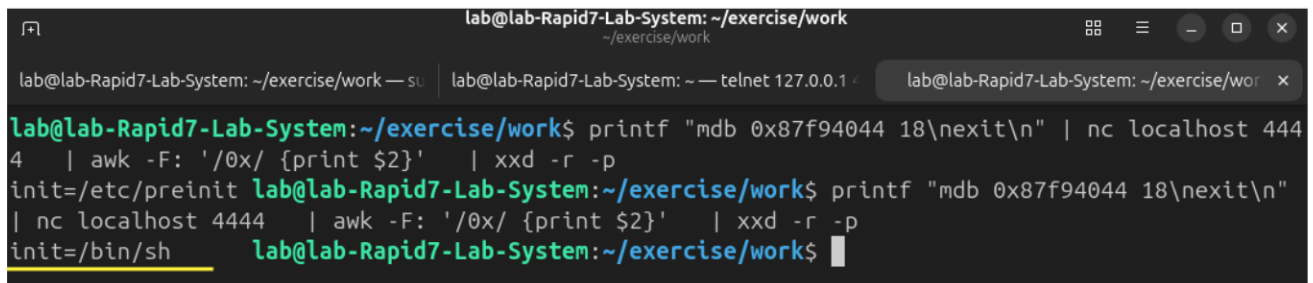
Figure 30: Hex to Ascii conversion of memory space

Step 4.4: Next, click on Terminal 2 the telnet terminal, and enter the following command, which will rewrite the Malloc RAM area starting at 0x87f9404a with etc/sh. Also note, since our changes are only 6 characters long and we are replacing **bin/preinit** which is 11 characters long, we are overwriting those extra characters with 0x20s, which is spaces.

```
write_memory 0x87F9404A 8 {0x62 0x69 0x6e 0x2f 0x73 0x68  
0x20 0x20 0x20 0x20 0x20}
```

Step 4.5: To validate that the above command correctly overwrote etc/preinit with bin/sh, rerun the following command in Terminal 3 like Step 4.3. To show the Ascii of the changes you made (Figure 31).

```
printf "mdb 0x87f94044 18\nexit\n" | nc localhost 4444 | awk -F:  
'/0x/ {print $2}' | xxd -r -p
```



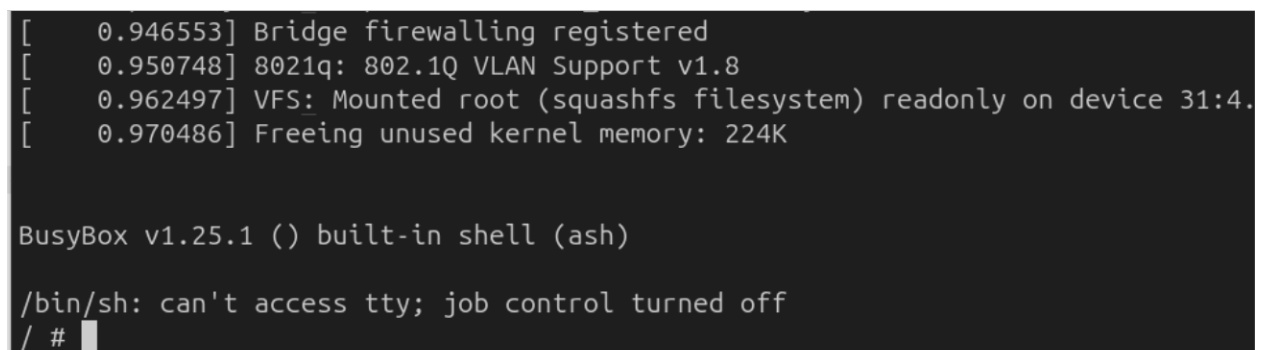
```
lab@lab-Rapid7-Lab-System: ~/exercise/work
lab@lab-Rapid7-Lab-System: ~/exercise/work — su — lab@lab-Rapid7-Lab-System: ~ — telnet 127.0.0.1 — lab@lab-Rapid7-Lab-System: ~/exercise/work
lab@lab-Rapid7-Lab-System:~/exercise/work$ printf "mdb 0x87f94044 18\nexit\n" | nc localhost 4444 | awk -F: '/0x/ {print $2}' | xxd -r -p
init=/etc/preinit lab@lab-Rapid7-Lab-System:~/exercise/work$ printf "mdb 0x87f94044 18\nexit\n" | nc localhost 4444 | awk -F: '/0x/ {print $2}' | xxd -r -p
init=/bin/sh lab@lab-Rapid7-Lab-System:~/exercise/work$
```

Figure 31: Hex to Ascii conversion of memory space

Step 4.6: If step 4.5 above returned the correct response of init=/bin/sh, then switch back over to Terminal 2 telnet , enter the following command, and hit enter.

```
resume
```

In the tio console, you should see the count down restart and system start booting. If everything is successful, the OpenMesh device should boot into single user mode. You should get the following tio console response (Figure 32) once the system is finished booting into single user mode.



```
[ 0.946553] Bridge firewalling registered
[ 0.950748] 8021q: 802.1Q VLAN Support v1.8
[ 0.962497] VFS: Mounted root (squashfs filesystem) readonly on device 31:4.
[ 0.970486] Freeing unused kernel memory: 224K

BusyBox v1.25.1 () built-in shell (ash)

/bin/sh: can't access tty; job control turned off
/ #
```

Figure 32: Single User Boot

W00t you now have root access, but not for long. The hardware watchdog has again started its counting down, and the system will panic and reboot in about 5 minutes. To prevent this from happening, you need to complete the following section. Or be forced to start over at Step 4.2 – Good Luck ;)

Remember, the following commands can be cut and pasted all at same time for each step using your mouse and “Command Notes” on the desktop – this saves time in typing.

Part 5 – Virtual file systems & Watchdog enable

I recommend not digging in deep by reading the *Note* parts in this section until after watchdog is started, so you won't have to keep redoing parts of this exercise. So first, just run the commands. Once you have watchdog started, you can then circle back on this section and read the *Note* writeups, which help explain what is going on.

Step 5.1: In the tio terminal, where you have root access to the OpenMesh device, enter the following commands to mount the needed virtual file system:

```
mount -t proc proc /proc  
mount -t sysfs sysfs /sys  
mount -t tmpfs tmpfs /tmp  
mount -t tmpfs tmpfs /dev
```

Note: To better understand the purpose and need for running the above commands, we have the following description. When the device is booted directly into single-user mode using init=/bin/sh, the normal Linux initialization process is bypassed. As a result, several essential virtual file systems which are normally mounted during boot are not available. Before additional applications and utilities can operate correctly, these file systems must be mounted manually.

The /proc file system provides a virtual interface to kernel and process information, allowing applications to query the current state of the operating system. The /sys file system exposes kernel objects, device information, and driver interfaces used by many system utilities. The /tmp file system is created as a temporary RAM-backed storage area where applications can create temporary files during execution. Finally, a new RAM-backed /dev file system is created to hold the device nodes required for communication with the kernel and hardware devices. Recreating these virtual file systems establishes the minimum operating environment needed for the remainder of the exercise while maintaining control of the system in a single-user shell.

Step 5.2: In the tio terminal, where you have root access to the OpenMesh device, enter the following commands to create the required device nodes for /dev/null:

```
mknod /dev/null c 1 3  
chmod 666 /dev/null
```

Note: The `mknod` command is used to manually create device nodes within the `/dev` directory. Unlike regular files, device nodes act as interfaces between user-space applications and device drivers within the Linux kernel. The general syntax is:

```
mknod <device_name> <type> <major_number> <minor_number>
```

For example, the command used in this exercise:

```
mknod /dev/null c 1 3
```

creates the `/dev/null` device node where:

`/dev/null` is the name of the device node being created.

`c` specifies that this is a character device, meaning data is transferred one character (byte) at a time. Block devices, such as hard drives and flash storage, use `b` instead.

`1` is the major device number, which identifies the kernel driver responsible for servicing the device.

`3` is the minor device number, which identifies a specific device or function managed by that driver.

In this case, the Linux kernel recognizes the major/minor combination `1:3` as the standard `/dev/null` device. Any data written to `/dev/null` is discarded, while any attempt to read from it immediately returns an end-of-file (EOF) condition. Because many Linux applications and shell scripts expect `/dev/null` to exist, recreating it after mounting a new `tmpfs` over `/dev` is an important step in rebuilding the operating environment.

Step 5.3: In the `ti0` terminal, where you have root access to the OpenMesh device, enter the following commands to start the watchdog process:

/sbin/om-watchdog 16 &

Note: The following description addresses the watchdog process. After booting into single-user mode, the device's normal initialization process has been bypassed, meaning background services that are typically started during the standard boot sequence are not running. One of these services is the hardware watchdog, whose purpose is to periodically signal the hardware that the operating system is functioning correctly. If these signals stop, the watchdog assumes the system has become unresponsive and automatically reboots the device after about 5 minutes.

During analysis, it was observed that the OpenMesh hardware consistently rebooted after a short period (5 minutes) while operating in single-user mode. To determine the cause, the normal OpenWrt boot process was examined by reviewing the system initialization scripts located in `/etc/preinit`, `/etc/inittab`, `/etc/init.d`, and the symbolic links under `/etc/rc.d`, which define the order in which services are

started during boot. These scripts identified the OpenMesh, specifically the /sbin/om-watchdog utility as the process responsible for servicing the hardware watchdog. Starting it manually with the command /sbin/om-watchdog 16 & restores the expected watchdog servicing, preventing the hardware from automatically rebooting.

Congratulations!!! Now that you have the system up and running, watchdog enabled, and system stable. **You no longer need to rush.** I recommend circling back and reading through the explanation *Notes* in section part 5 before continuing the remainder of exercise.

Part 6 – Enabling Wired Network Communication

In this section, you will focus on enabling network services. Here, you will manually load the required kernel driver, configure the wired Ethernet interface with a static IP address, and verify network connectivity. Once the network interface is operational, we will start the Dropbear SSH server, a lightweight SSH server commonly used on embedded Linux systems such as OpenWrt. This will allow you to remotely access the device from your laptop using an SSH client, demonstrating that the operating environment has been restored sufficiently to support network-based administration while remaining in the temporary environment.

Note: During a normal boot, the Linux kernel automatically loads the hardware drivers required by the system, including the onboard Ethernet driver, and creates the eth network interfaces. Because this exercise bypasses the normal initialization process by booting directly into /bin/sh, these drivers are never loaded and the network interfaces are not available. By observing the console output during a normal boot process, in tio serial terminal, the ethernet ag71xx can be observed being loaded. This allows for the identification of the kernel module responsible for initializing the onboard Ethernet controller.

Step 6.1: Using the Linux command **insmod**, we can load the ethernet kernel module into memory. To accomplish this, run the following command on the Mesh WiFi device in the tio console. The output of this command is shown below in Figure 33:

insmod /lib/modules/4.4.184/ag71xx.ko

```
/ # insmod /lib/modules/4.4.184/ag71xx.ko
[ 42.893769] libphy: ag71xx_mdio: probed
[ 43.560481] ag71xx ag71xx.0: connected to PHY at ag71xx-mdio.0:01 [uid=004dd072, driver=Atheros 8035 ethernet]
[ 43.571386] eth0: Atheros AG71xx at 0xb9000000, irq 4, mode:RGMII
[ 44.240910] ag71xx ag71xx.1: connected to PHY at ag71xx-mdio.0:02 [uid=004dd074, driver=Atheros 8031/8033 ethernet]
[ 44.252245] eth1: Atheros AG71xx at 0xba000000, irq 5, mode:SGMII
/ #
```

Figure 33: Loading Ethernet Kernel Driver

Step 6.2: Once the Ethernet kernel module is loaded, we can next assign an IP address and enable the interface. To accomplish this, run the following command in the tio console. The response should look similar to Figure 34:

ifconfig eth1 10.10.10.101 netmask 255.255.255.0 up

```

ifconfig eth1 10.10.10.101 netmask 255.255.255.0 up
[ 86.963284] IPv6: ADDRCONF(NETDEV_UP): eth1: link is not ready
/ # [ 90.241454] Resetting SGMII link
[ 90.244852] state 0x1
[ 90.257218] Reached count 1
[ 90.260055] eth1: link up (1000Mbps/Full duplex)
[ 90.264769] IPv6: ADDRCONF(NETDEV_CHANGE): eth1: link becomes ready

```

Figure 34: Enable eth1 Assign IP Address

Step 6.3: Once the above command is run, you can next run the **ifconfig** command without any switches to show the available Ethernet interfaces and their status. The output of the following command is shown in Figure 35:

ifconfig

```

/ # ifconfig
eth1      Link encap:Ethernet  HWaddr AC:86:74:B1:EE:61
          inet addr:10.10.10.101  Bcast:10.10.10.255  Mask:255.255.255.0
          inet6 addr: fe80::ae86:74ff:feb1:ee61/64  Scope:Link
          inet6 addr: fe80::ae86:74ff:feb1:ee61/64  Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1153 errors:0 dropped:0 overruns:0 frame:0
          TX packets:746 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:195722 (191.1 KiB)  TX bytes:136430 (133.2 KiB)
          Interrupt:5

```

Figure 35: ifconfig Command

Step 6.4: You can validate this ethernet interface is working by running the following command in the tio console on the OpenMesh device, to ping your laptop at IP address 10.10.10.10:

Make sure you run the ping command with the -c switch, which sends 4 pings and then exits. If you do not do this the /bin/sh shell in the tio console lacks the ability to break out of a running application. This basically locks you out of your tio console and will require you to restart or end the exercise.

ping -c 4 10.10.10.10

Part 7 – Setting up Dropbear for SSH access

In this section, we will set up the runtime environment to support Dropbear SSH server services.

Step 7.1: As stated several times during this exercise, when the OpenMesh device was booted into single-user mode, the normal OpenWrt initialization sequence was bypassed. As a result, several device nodes and the pseudo-terminal (PTY) subsystems required by network services were never created. During our testing, attempts to start the Dropbear SSH server produced multiple errors indicating that pseudo terminals and random devices were unavailable. By examining these error messages generated during the trial & error phase of building this exercise, we were able to identify the missing device nodes and terminal subsystems required for running Dropbear SSH services. To fix these issues, the following commands need to be executed within the tio console to set up the needed runtime environment to start Dropbear. These commands can be cut and pasted from the [Command Notes](#) file. Also, these commands

have little or no output, so nothing to see. More detail on the mknod command and its purpose is covered in Notes: under Step 5.2.

```
mkdir -p /dev/pts  
mount -t devpts devpts /dev/pts  
mknod /dev/random c 1 8  
mknod /dev/urandom c 1 9  
chmod 666 /dev/random /dev/urandom  
mknod /dev/ptmx c 5 2  
chmod 666 /dev/ptmx  
mknod /dev/tty c 5 0  
chmod 666 /dev/tty
```

Step 7.2: Next, we need to generate a temporary SSH Host Key before starting Dropbear. An SSH server requires a host key to identify itself to connecting clients. Under normal operation, this key is stored in persistent flash. Because this exercise intentionally avoids modifying flash memory, a temporary RSA host key is generated and stored in the RAM-backed */tmp* filesystem. This provides the minimum requirement for secure SSH communications, while ensuring the device returns to its original state after reboot. To generate the needed RSA key, run the following command. These commands can be cut and pasted from the [Command Notes](#) file. The output from this command should look similar to Figure 36:

```
mkdir -p /tmp/dropbear  
dropbearkey -t rsa -f /tmp/dropbear/dropbear_rsa_host_key
```

```
/ # dropbearkey -t rsa -f /tmp/dropbear/dropbear_rsa_host_key  
Generating key, this may take a while...  
Public key portion is:  
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCDCCcifs1LrTcqaHIs/sYFM2Kyral/fyvUHkr  
Jak1Hi000T5qhBZ9FBK330EDwqgW4lxGSKMZ4nIrD55HkXRgDkl2bWAs5I9kUjKkqAELcW1Re2  
fzPmbqznzv/jDQeEwwCAXABd/JZRxo9M9a5YHIexeLZZY6nNqi4Qg/mR/Hn3PM3a2niF8I6W5N7  
Jo6tgyvVln+okDw+aYYNblf3I5RLJvSY5LAWs8Gqe26tL/UMh8F3uyddqvfbV6efVVMCD387AR  
FPPylzXRQZ39A7NAuchIIEJZbDXS+wBetpXFdpku2loewm1ckHEC094ZUfws2UmAfoEi4fHC9L  
NrEtNXj4PX root@(none)  
Fingerprint: sha1!! 4e:a7:7c:36:0e:bb:f5:52:35:9a:a6:c7:5f:5a:b5:a3:0f:bc:  
a2:94
```

Figure 36: RSA Key Generation

Step 7.3: Now that the needed RSA key has been generated, we can start Dropbear. Dropbear is a lightweight SSH server commonly used on embedded Linux systems such as OpenWrt. After the runtime environment has been restored, and a temporary RSA host key has been created, Dropbear can be started manually. The Dropbear service is configured to use the temporary RSA host key, and is launched as a background process, so the serial console remains available while accepting incoming SSH connections. To start Dropbear, run the

following command within the `ti` terminal. This command can be cut and pasted from the Command Notes file. The output from this command is minimal stating only “Running in background”. This is shown below in Figure 37:

```
dropbear -E -r /tmp/dropbear/dropbear_rsa_host_key -p 22 &
```

```
/ # dropbear -E -r /tmp/dropbear/dropbear_rsa_host_key -p 22 &  
/ # [349] Jan 01 00:03:21 Running in background
```

Figure 37: Dropbear Starting

Step 7.4: Now that we have Dropbear running on the OpenMesh device, we can ssh from our laptop to connect over ethernet to the OpenMesh device. Also note, because the version of Dropbear included with this firmware on this device only supports RSA host keys, our SSH client must temporarily enable compatibility mode. On your laptop, run the following command. These commands can be cut and pasted from the Command Notes file.

```
ssh -oHostKeyAlgorithms=+ssh-rsa -  
oPubkeyAcceptedAlgorithms=+ssh-rsa root@10.10.10.101
```

After running command if prompted for a yes/no response request just enter yes and hit return. Successfully establishing an SSH connection to the OpenMesh device should look like Figure 38:

```
lab@lab-Rapid7-Lab-System: ~/exercise/work — sudo lab@lab-Rapid7-Lab-System: ~/exercise/work — telnet lab@lab-Rapid7-Lab-System: ~/exercise/work — ssh
```

```
BusyBox v1.25.1 () built-in shell (ash)
```

```

  _ _ _ _ _
 /_/_/_/_/_\
|  LE  |
|  DE  |
|_/_/_/_/_|
|  LE  |
|  DE  |
|_/_/_/_/_|
  _ _ _ _ _

  _ _ _ _ _
 | | | | |
 | | | | |
 | | | | |
 | | | | |

lede-project.org

-----
Reboot (17.01-SNAPSHOT, r0+4183-db16951)
-----

=== WARNING! =====
There is no root password defined on this device!
Use the "passwd" command to set up a new password
in order to prevent unauthorized SSH logins.
=====

root@(none):~#
```

Figure 38: SSH Connection to OpenMesh

Congratulations!!! you have completed the exercise. Feel free to take a few minutes to explore the system further and ask questions.

Conclusion:

Thank you very much for participating in today's exercise. If you have access to **linkedin** you could help us out greatly by posting positive feedback about this exercise to **@Rapid7** and **@Deral Heiland**

Do not unplug anything on the exercise or laptop. Just turn off the power switch to OpenMesh device and on the laptop double click the reset Exercise Icon on the desktop.

